

ACN Tutorial: Reliable File Transfer over TCP

Multi TCP Connections vs Single TCP Connection

Department of Computer Science and Engineering

Name - Atish Kadam

Roll – CS25MTECH14003

Objective

The objective of this experiment is to compare the performance of Single TCP Connection and Multiple TCP Connections for reliable file transfer over TCP.

Experimental Setup

- TCP Connections:
 - Single TCP: One connection for complete file transfer
 - Multi TCP: Four concurrent TCP connections handling different chunks simultaneously
- Metrics Observed using Wireshark:
 - Number of packets sent
 - Retransmissions (if any)
 - Time taken for file transfer
- Environment:
 - Operating System – Windows 11
 - Programming Language: Python
 - Tool used for tracing packets: Wireshark.

Procedure

For Single TCP Connection:

1. Run Server by entering “python server.py” on terminal.
2. Run Client by entering “python client.py” on terminal.
3. You will see provide path to file to be upload.
4. Add path of file.

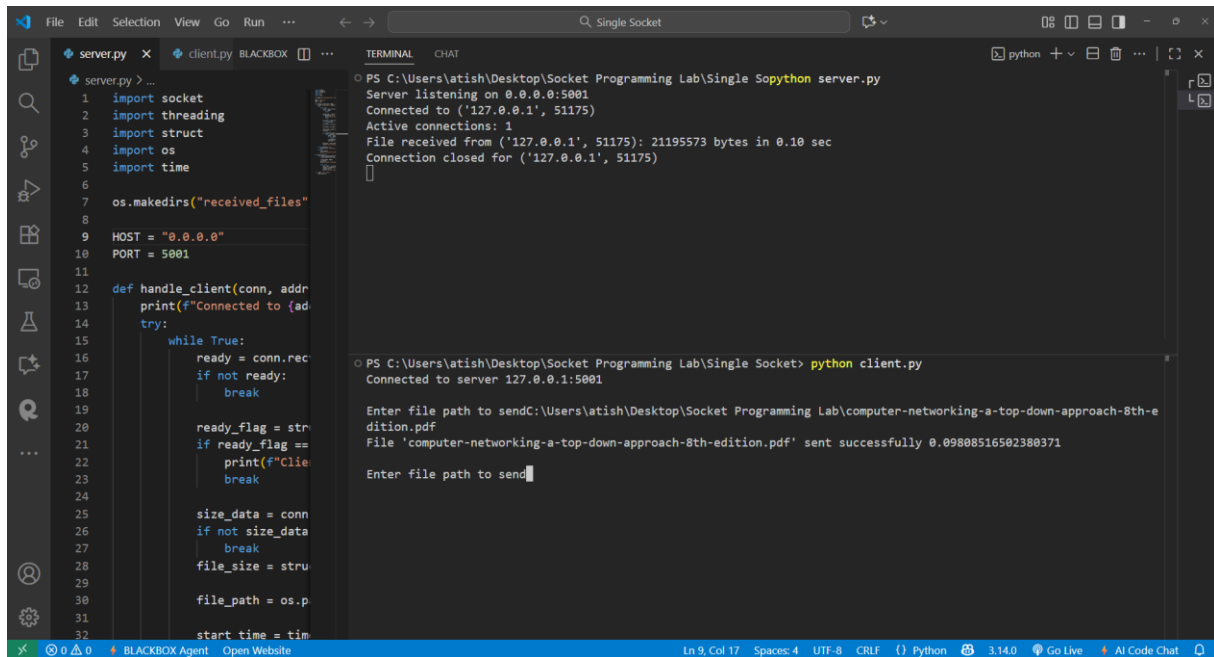
For Multiple TCP Connection:

1. Run Server by entering “python server.py” on terminal.
2. Run Client by entering “python client.py” on terminal.
3. You will see provide path to file to be upload.
4. Add path then you can see total 4 parallel connections set.

Experimental Result

Single TCP Connection

Iteration - 1



```
server.py
1 import socket
2 import threading
3 import struct
4 import os
5 import time
6
7 os.makedirs("received_files")
8
9 HOST = "0.0.0.0"
10 PORT = 5001
11
12 def handle_client(conn, addr):
13     print(f"Connected to {addr}")
14     try:
15         while True:
16             ready = conn.recv(1024)
17             if not ready:
18                 break
19
20             ready_flag = struct.unpack('i', ready)[0]
21             if ready_flag == 1:
22                 print(f"Client {addr} is ready to send")
23                 break
24
25             size_data = conn.recv(1024)
26             if not size_data:
27                 break
28             file_size = struct.unpack('i', size_data)[0]
29             file_path = os.path.join("received_files", f"{addr}_{file_size}.pdf")
30             with open(file_path, "wb") as f:
31                 f.write(ready)
32             print(f"File received from {addr}: {file_size} bytes in 0.10 sec")
33             connection_closed = conn.recv(1024)
34             if connection_closed:
35                 print(f"Connection closed for {addr}")
36
37     except Exception as e:
38         print(f"Error: {e}")
39
40 if __name__ == "__main__":
41     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
42     server.bind((HOST, PORT))
43     server.listen(5)
44     print("Server listening on 0.0.0.0:5001")
45     while True:
46         conn, addr = server.accept()
47         thread = threading.Thread(target=handle_client, args=(conn, addr))
48         thread.start()
49         print(f"Active connections: 1")
50
51 client.py
1 import socket
2
3 host = "127.0.0.1"
4 port = 5001
5
6 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 s.connect((host, port))
8
9 print(f"Connected to server {host}:{port}")
10
11 file_path = "computer-networking-a-top-down-approach-8th-edition.pdf"
12 with open(file_path, "rb") as f:
13     data = f.read(1024)
14     s.sendall(struct.pack('i', 1) + data)
15
16 print(f"File '{file_path}' sent successfully 0.09808516502380371")
17
18 file_path = input("Enter file path to send: ")
19
```

Terminal Output:

```
PS C:\Users\atish\Desktop\Socket Programming Lab\Single Sopython> python server.py
Server listening on 0.0.0.0:5001
Connected to ('127.0.0.1', 51175)
Active connections: 1
File received from ('127.0.0.1', 51175): 21195573 bytes in 0.10 sec
Connection closed for ('127.0.0.1', 51175)

PS C:\Users\atish\Desktop\Socket Programming Lab\Single Socket> python client.py
Connected to server 127.0.0.1:5001
Enter file path to sendC:\Users\atish\Desktop\Socket Programming Lab\computer-networking-a-top-down-approach-8th-edition.pdf
File 'computer-networking-a-top-down-approach-8th-edition.pdf' sent successfully 0.09808516502380371
Enter file path to send
```

Wireshark - Conversations - Single_1.pcapng

Conversation Settings

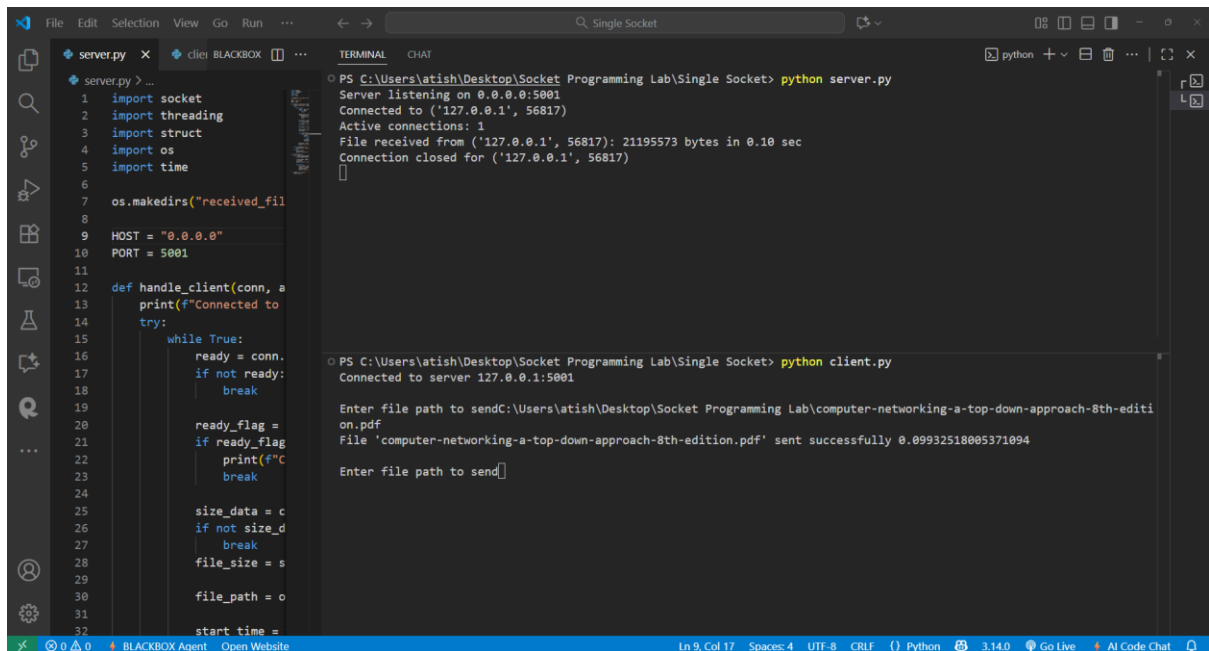
Name resolution

Absolute start time

Limit to display filter

Ethernet	IPv4 - 1	IPv6	TCP - 2	UDP											
Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	Flows
127.0.0.1	61606	127.0.0.1	64098	4	188 bytes	1	2	97 bytes	2	91 bytes	1.469434	0.0012			2
127.0.0.1	51175	127.0.0.1	5001	10,307	22 MB	0	5,152	21 MB	5,155	227 kB	0.000000	10.4124	16 Mbps	174 kbps	1

Iteration – 2



```
server.py
1 import socket
2 import threading
3 import struct
4 import os
5 import time
6
7 os.makedirs("received_files")
8
9 HOST = "0.0.0.0"
10 PORT = 5001
11
12 def handle_client(conn, addr):
13     print(f"Connected to {addr}")
14     try:
15         while True:
16             ready = conn.recv(1024)
17             if not ready:
18                 break
19
20             ready_flag = struct.unpack('i', ready)[0]
21             if ready_flag == 1:
22                 print(f"Client {addr} is ready to send")
23                 break
24
25             size_data = conn.recv(1024)
26             if not size_data:
27                 break
28             file_size = struct.unpack('i', size_data)[0]
29             file_path = os.path.join("received_files", f"{addr}_{file_size}.pdf")
30             with open(file_path, "wb") as f:
31                 f.write(ready)
32             print(f"File received from {addr}: {file_size} bytes in 0.10 sec")
33             connection_closed = conn.recv(1024)
34             if connection_closed:
35                 print(f"Connection closed for {addr}")
36
37     except Exception as e:
38         print(f"Error: {e}")
39
40 if __name__ == "__main__":
41     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
42     server.bind((HOST, PORT))
43     server.listen(5)
44     print("Server listening on 0.0.0.0:5001")
45     while True:
46         conn, addr = server.accept()
47         thread = threading.Thread(target=handle_client, args=(conn, addr))
48         thread.start()
49         print(f"Active connections: 1")
50
51 client.py
1 import socket
2
3 host = "127.0.0.1"
4 port = 5001
5
6 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 s.connect((host, port))
8
9 print(f"Connected to server {host}:{port}")
10
11 file_path = "computer-networking-a-top-down-approach-8th-edition.pdf"
12 with open(file_path, "rb") as f:
13     data = f.read(1024)
14     s.sendall(struct.pack('i', 1) + data)
15
16 print(f"File '{file_path}' sent successfully 0.09932518005371094")
17
18 file_path = input("Enter file path to send: ")
19
```

Terminal Output:

```
PS C:\Users\atish\Desktop\Socket Programming Lab\Single Socket> python server.py
Server listening on 0.0.0.0:5001
Connected to ('127.0.0.1', 56817)
Active connections: 1
File received from ('127.0.0.1', 56817): 21195573 bytes in 0.10 sec
Connection closed for ('127.0.0.1', 56817)

PS C:\Users\atish\Desktop\Socket Programming Lab\Single Socket> python client.py
Connected to server 127.0.0.1:5001
Enter file path to sendC:\Users\atish\Desktop\Socket Programming Lab\computer-networking-a-top-down-approach-8th-edition.pdf
File 'computer-networking-a-top-down-approach-8th-edition.pdf' sent successfully 0.09932518005371094
Enter file path to send
```

Wireshark - Conversations - Single_2_capture

Conversation Settings

Name resolution

☒ Absolute start time

☒ Limit to display filter

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	Flows
127.0.0.1	56817	127.0.0.1	5001	10,310	22 MB	0	5,152	21 MB	5,158	227 kB	0.000000	20.0153	8562 kbps	90 kbps	1
127.0.0.1	61606	127.0.0.1	5001	4	188 bytes	1	2	97 bytes	2	91 bytes	15.480738	0.0009			2

Iteration – 3

The screenshot shows a code editor with two files: `server.py` and `client.py`. The `server.py` file contains the following code:

```
1 import socket
2 import threading
3 import struct
4 import os
5 import time
6
7 os.makedirs("received_files", exist_ok=True)
8
9 HOST = "0.0.0.0"
10 PORT = 5001
11
12 def handle_client(conn, addr):
13     print(f"Connected to {addr}")
14     try:
15         while True:
16             ready = conn.recv(4)
17             if not ready:
18                 break
19
20             ready_flag = struct.unpack("I", ready)
21             if ready_flag == 0:
22                 print(f"Client {addr} disconnected")
23                 break
24
25             size_data = conn.recv(8)
26             if not size_data:
27                 break
28             file_size = struct.unpack("I", size_data)
29             file_path = os.path.join("received_files", f"{addr}_{file_size}.pdf")
30             with open(file_path, "wb") as f:
31                 f.write(ready)
32             start_time = time.time()
```

The `client.py` file contains the following code:

```
1 import socket
2
3 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4 s.connect(("127.0.0.1", 5001))
5
6 file_path = input("Enter file path to send: ")
7 with open(file_path, "rb") as f:
8     data = f.read(1024)
9     while data:
10         s.sendall(data)
11         data = f.read(1024)
12
13 s.close()
```

The terminal output shows the server listening on 0.0.0.0:5001, connected to 127.0.0.1:49975, and receiving a file. The client output shows it connected to the server and sent the file successfully.

Wireshark - Conversations - Single_Socketing

Conversation Settings	Ethernet II (IPv4-2)	IPv6-1	TCP-2	UDP-2
Name resolution	Address A - 127.0.0.1	Port B - 5001	Packets - 3001	Bytes - 18332
Absolute start time	127.0.0.1	61606	127.0.0.1	64098
Limit to display filter				4 180 bytes

Multiple

Iteration – 1

The screenshot shows a code editor with two files: `client.py` and `server.py`. The `server.py` file contains the following code:

```
1 import socket
2 import threading
3 import struct
4 import os
5 import time
6
7 SERVER_IP = "127.0.0.1"
8 SERVER_PORT = 5005
9 CHUNK_SIZE = 1024 * 1024
10 MAX_CONNECTIONS = 4
11
12 thread_end_times = []
13 lock = threading.Lock()
14
15 def send_chunk(chunk_index, total_chunks, chunk_data):
16     global thread_end_times
17     try:
18         s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
19         s.connect((SERVER_IP, SERVER_PORT))
20         header = struct.pack("I", chunk_index, total_chunks, len(chunk_data))
21         s.sendall(header)
22         s.sendall(chunk_data)
23         s.shutdown(socket.SHUT_WR)
24         s.close()
25     except Exception as e:
26         print(f"Error sending chunk {chunk_index}: {e}")
27
28     with lock:
29         thread_end_times.append(time.time())
30
31     print(f"Sent chunk {chunk_index} / {total_chunks}")
32
33 def main():
34     global thread_end_times
35     file_path = input("Enter file path to send: ")
36     if not os.path.exists(file_path):
37         print("File not found.")
38         return
39
40     file_size = os.path.getsize(file_path)
41     total_chunks = math.ceil(file_size / CHUNK_SIZE)
42     print(f"Splitting {file_path} into {total_chunks} chunks...")
43
44     with open(file_path, "rb") as f:
45         chunks = [f.read(CHUNK_SIZE) for _ in range(total_chunks)]
46
47     threads = []
48     for i in range(total_chunks):
49         t = threading.Thread(target=send_chunk, args=(i, total_chunks, chunks[i]))
50         threads.append(t)
51         t.start()
52
53     for t in threads:
54         t.join()
55
56     print(f"File received in {thread_end_times[-1] - thread_end_times[0]} seconds.")
```

The `client.py` file contains the following code:

```
1 import socket
2
3 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4 s.connect(("127.0.0.1", 5005))
5
6 file_path = input("Enter file path to send: ")
7 with open(file_path, "rb") as f:
8     data = f.read(1024)
9     while data:
10         s.sendall(data)
11         data = f.read(1024)
12
13 s.close()
```

The terminal output shows the server listening on 0.0.0.0:5005, receiving chunks from 127.0.0.1:50065, and sending them to 127.0.0.1:50065. The client output shows it connected to the server and sent the file successfully.

Wireshark - Conversations - Multiple_Sockets

Conversation Settings	Ethernet II (IPv4-2)	IPv6-1	TCP-2	UDP-2
Name resolution	Address A - 127.0.0.1	Port B - 5005	Packets - 3001	Bytes - 18332
Absolute start time	127.0.0.1	61606	127.0.0.1	64098
Limit to display filter				4 180 bytes

Wireshark - Conversations - Multiple Tcpdump

Conversation Settings

Name resolution: ☐ Absolute start time: ☐ Limit to display filter: ☒

Copy Follow Stream... Graph...

Ethernet	IPv4	IPv6	TCP	UDP	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	Flow
127.0.0.1	49693	127.0.0.1	5005	32	2 KB	0	16	776 bytes	16	728 bytes	0.000000	175.0715	35 bits/s	33 bits/s	16
127.0.0.1	50065	127.0.0.1	5005	48	1 MB	2	21	1 MB	27	1 MB	6.638132	0.1203	63 Mbps	78 Mbps	2
127.0.0.1	50066	127.0.0.1	5005	39	1 MB	1	21	1 MB	18	804 bytes	6.638131	0.2807	104 Mbps	79 Mbps	1
127.0.0.1	50067	127.0.0.1	5005	48	1 MB	3	21	1 MB	27	1 MB	6.638132	0.2809	21 Mbps	22 Mbps	1
127.0.0.1	50068	127.0.0.1	5005	55	1 MB	4	21	1 MB	34	2 MB	6.638132	0.3184	26 Mbps	27 Mbps	1
127.0.0.1	50069	127.0.0.1	5005	48	1 MB	5	21	1 MB	27	1 MB	6.638127	0.2407	33 Mbps	38 Mbps	1
127.0.0.1	50070	127.0.0.1	5005	48	1 MB	6	21	1 MB	27	1 MB	6.638132	0.2500	28 Mbps	23 Mbps	1
127.0.0.1	50071	127.0.0.1	5005	48	1 MB	7	21	1 MB	27	1 MB	6.638134	0.4546	18 Mbps	21 Mbps	1
127.0.0.1	50072	127.0.0.1	5005	48	1 MB	8	21	1 MB	27	1 MB	6.638134	0.5120	16 Mbps	23 Mbps	1
127.0.0.1	50073	127.0.0.1	5005	48	1 MB	9	21	1 MB	27	1 MB	6.640570	0.5495	15 Mbps	17 Mbps	1
127.0.0.1	50074	127.0.0.1	5005	48	1 MB	10	21	1 MB	27	1 MB	6.640753	0.5941	11 Mbps	13 Mbps	1
127.0.0.1	50075	127.0.0.1	5005	48	1 MB	11	21	1 MB	27	1 MB	6.641055	0.6461	12 Mbps	13 Mbps	1
127.0.0.1	50076	127.0.0.1	5005	48	1 MB	12	21	1 MB	27	1 MB	6.641147	0.7006	10 Mbps	12 Mbps	1
127.0.0.1	50077	127.0.0.1	5005	48	1 MB	13	21	1 MB	27	1 MB	6.642040	0.7499	13 Mbps	13 Mbps	1
127.0.0.1	50078	127.0.0.1	5005	48	1 MB	14	21	1 MB	27	1 MB	6.643029	0.7312	11 Mbps	13 Mbps	1
127.0.0.1	50079	127.0.0.1	5005	48	1 MB	15	21	1 MB	27	1 MB	6.643224	0.7251	11 Mbps	13 Mbps	1
127.0.0.1	50080	127.0.0.1	5005	48	1 MB	16	21	1 MB	27	1 MB	6.643314	0.8005	10 Mbps	11 Mbps	1
127.0.0.1	50081	127.0.0.1	5005	48	1 MB	17	21	1 MB	27	1 MB	6.643365	0.7817	10 Mbps	12 Mbps	1
127.0.0.1	50082	127.0.0.1	5005	57	1 MB	18	22	1 MB	35	2 MB	6.643776	0.7832	10 Mbps	13 Mbps	1
127.0.0.1	50083	127.0.0.1	5005	50	1 MB	19	22	1 MB	28	1 MB	6.648077	0.8302	10 Mbps	12 Mbps	1
127.0.0.1	50084	127.0.0.1	5005	59	1 MB	20	22	1 MB	28	1 MB	6.648249	0.8875	10 Mbps	12 Mbps	1
127.0.0.1	50085	127.0.0.1	5005	17	225 KB	21	9	224 KB	8	364 bytes	6.648230	0.4504	3917 kbps	6352 kbps	1

Iteration – 2

File Edit Selection View Go Run Terminal Help

server.py

```

59 if total_chunks_expected and len(received_chunks) == total_chunks:
60     if time.time() - last_data_time < 0.5:
61         continue
62     total_time = (last_data_time - first_data_time) if first_data_time else 0
63     print(f"File received in {total_time:.3f} seconds.\n")
64     with open(OUTPUT_FILE, "wb") as f:
65         for i in range(total_chunks_expected):
66             f.write(received_chunks[i])
67     print(f"File assembled successfully - {OUTPUT_FILE}\n")
68     # Reset
69     received_chunks.clear()
70     total_chunks_expected = None
71     first_data_time = None
72     last_data_time = None
73
74 def start_server():
75     print(f"Server listening on {HOST}:{PORT}")
76     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
77         s.bind((HOST, PORT))
78         s.listen()
79         while True:
80             conn, addr = s.accept()
81             threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()
82
83 if __name__ == '__main__':
84     start_server()
85
86
87
88
89
90
91

```

Multiple Socket

PS C:\Users\lath\Desktop\Socket Programming Lab\Multiple Sockets> python server.py

```

Received chunk 4/21 from ('127.0.0.1', 58657)
Received chunk 1/21 from ('127.0.0.1', 58651)
Received chunk 21/21 from ('127.0.0.1', 58652)
Received chunk 3/21 from ('127.0.0.1', 58652)
Received chunk 6/21 from ('127.0.0.1', 58655)
Received chunk 5/21 from ('127.0.0.1', 58654)
Received chunk 7/21 from ('127.0.0.1', 58656)
Received chunk 11/21 from ('127.0.0.1', 58660)
Received chunk 9/21 from ('127.0.0.1', 58653)
Received chunk 16/21 from ('127.0.0.1', 58659)
Received chunk 18/21 from ('127.0.0.1', 58660)
Received chunk 13/21 from ('127.0.0.1', 58662)
Received chunk 14/21 from ('127.0.0.1', 58663)
Received chunk 12/21 from ('127.0.0.1', 58664)
Received chunk 15/21 from ('127.0.0.1', 58664)
Received chunk 20/21 from ('127.0.0.1', 58669)
Received chunk 17/21 from ('127.0.0.1', 58667)
Received chunk 15/21 from ('127.0.0.1', 58667)
File received in 0.782 seconds.
File assembled successfully - received_files\final_received_file.pdf

```

PS C:\Users\lath\Desktop\Socket Programming Lab\Multiple Sockets> python client.py

```

Starting upload with 4 connections...
Sent chunk 2/21
Sent chunk 1/21
Sent chunk 4/21
Sent chunk 3/21
Sent chunk 5/21
Sent chunk 6/21
Sent chunk 7/21
Sent chunk 9/21
Sent chunk 10/21
Sent chunk 11/21
Sent chunk 12/21
Sent chunk 13/21
Sent chunk 14/21
Sent chunk 15/21
Sent chunk 16/21
Sent chunk 17/21
Sent chunk 18/21
Sent chunk 19/21
Sent chunk 20/21
Sent chunk 21/21
File 'computer-networking-a-top-down-approach-8th-edition.pdf' sent successfully.

```

Wireshark - Conversations - Multiple Tcpdump

Conversation Settings

Name resolution: ☐ Absolute start time: ☐ Limit to display filter: ☒

Copy Follow Stream... Graph...

Protocol: Bluetooth, BAC, DCCP, Ethernet, FC

Ethernet	IPv4	IPv6	TCP	UDP	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	Flow
127.0.0.1	49693	127.0.0.1	5005	8	376 bytes	0	4	194 bytes	4	182 bytes	0.000000	25.0168	62 bits/s	58 bits/s	4
127.0.0.1	58659	127.0.0.1	5005	48	1 MB	1	21	1 MB	27	1 MB	15.387321	0.0448	80 Mbps	114 Mbps	1
127.0.0.1	58651	127.0.0.1	5005	48	1 MB	2	21	1 MB	27	1 MB	15.387531	0.1608	52 Mbps	58 Mbps	1
127.0.0.1	58652	127.0.0.1	5005	48	1 MB	3	21	1 MB	27	1 MB	15.387640	0.2601	32 Mbps	36 Mbps	1
127.0.0.1	58653	127.0.0.1	5005	48	1 MB	4	21	1 MB	27	1 MB	15.387850	0.1150	62 Mbps	72 Mbps	1
127.0.0.1	58654	127.0.0.1	5005	48	1 MB	5	21	1 MB	27	1 MB	15.388527	0.3670	22 Mbps	26 Mbps	1
127.0.0.1	58659	127.0.0.1	5005	48	1 MB	6	21	1 MB	27	1 MB	15.388714	0.1339	25 Mbps	28 Mbps	1
127.0.0.1	58656	127.0.0.1	5005	48	1 MB	7	21	1 MB	27	1 MB	15.388965	0.5018	16 Mbps	19 Mbps	1
127.0.0.1	58657	127.0.0.1	5005	48	1 MB	8	21	1 MB	27	1 MB	15.390174	0.5003	16 Mbps	19 Mbps	1
127.0.0.1	58659	127.0.0.1	5005	48	1 MB	9	21	1 MB	27	1 MB	15.391348	0.5451	13 Mbps	17 Mbps	1
127.0.0.1	58659	127.0.0.1	5005	48	1 MB	10	21	1 MB	27	1 MB	15.391864	0.6759	12 Mbps	14 Mbps	1
127.0.0.1	58660	127.0.0.1	5005	48	1 MB	11	21	1 MB	27	1 MB	15.392279	0.5399	15 Mbps	17 Mbps	1
127.0.0.1	58661	127.0.0.1	5005	48	1 MB	12	21	1 MB	27	1 MB	15.392602	0.7642	10 Mbps	12 Mbps	1
127.0.0.1	58662	127.0.0.1	5005	48	1 MB	13	21	1 MB	27	1 MB	15.394035	0.7389	11 Mbps	12 Mbps	1
127.0.0.1	58669	127.0.0.1	5005	55	1 MB	14	21	1 MB	34	2 MB	15.394444	0.7398	11 Mbps	16 Mbps	1
127.0.0.1	58664	127.0.0.1	5005	48	1 MB	15	21	1 MB	27	1 MB	15.394941	0.7682	10 Mbps	12 Mbps	1
127.0.0.1	58665	127.0.0.1	5005	48	1 MB	16	21	1 MB	27	1 MB	15.394745	0.5800	14 Mbps	16 Mbps	1
127.0.0.1	58666	127.0.0.1	5005	48	1 MB	17	21	1 MB	27	1 MB	15.395892	0.7732	10 Mbps	12 Mbps	1
127.0.0.1	58667	127.0.0.1	5005	48	1 MB	18	21	1 MB	27	1 MB	15.397551	0.7754	10 Mbps	12 Mbps	1
127.0.0.1	58668	127.0.0.1	5005	48	1 MB	19	21	1 MB	27	1 MB	15.397678	0.7055	11 Mbps	13 Mbps	1
127.0.0.1	58669	127.0.0.1	5005	48	1 MB	20	21	1 MB	27	1 MB	15.397823	0.7667	10 Mbps	12 Mbps	1
127.0.0.1	58670	127.0.0.1	5005	17	225 KB	21	9	224 KB	8	364 bytes	15.398858	0.1870	9601 kbps	15 kbps	1

Iteration – 3

```

server.py
def assemble_file():
    if total_chunks_expected and len(received_chunks) == total_chunks_expected:
        if time.time() - last_data_time < 0.5:
            continue
        total_time = (last_data_time - first_data_time) if first_data_time is not None else 0
        print(f"File received in {total_time:.2f} seconds.\n")
        with open(OUTPUT_FILE, "wb") as f:
            for i in range(total_chunks_expected):
                f.write(received_chunks[i])
        print(f"File assembled successfully - {OUTPUT_FILE}\n")
        # Reset
        received_chunks.clear()
        total_chunks_expected = None
        first_data_time = None
        last_data_time = None

def start_server():
    print(f"Server listening on {HOST}:{PORT}")
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((HOST, PORT))
        s.listen()
        while True:
            conn, addr = s.accept()
            threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()

if __name__ == "__main__":
    start_server()

client.py
def start_client():
    print(f"Client connecting to {HOST}:{PORT}")
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        while True:
            chunk = f.read(CHUNK_SIZE)
            if not chunk:
                break
            s.send(chunk)
    print(f"File sent successfully - {INPUT_FILE}\n")

if __name__ == "__main__":
    start_client()
    
```

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel. Start	Duration	Bits A → B	Bits B → A	Flow
127.0.0.1	4993	127.0.0.1	5005	8	376 bytes	0	4	194 bytes	4	182 bytes	0.000000	25.0068	62 bits/s	58 bits/s	4
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	1	21	1 MB	27	1 MB	25.20345	0.1106	75 Mbps	86 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	2	21	1 MB	27	1 MB	25.20345	0.1106	75 Mbps	86 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	3	21	1 MB	27	1 MB	25.206165	0.2653	31 Mbps	36 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	4	21	1 MB	27	1 MB	25.206165	0.2653	31 Mbps	36 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	5	21	1 MB	27	1 MB	25.207915	0.1992	52 Mbps	60 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	6	21	1 MB	27	1 MB	25.208051	0.2488	33 Mbps	38 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	7	21	1 MB	27	1 MB	25.208320	0.2782	30 Mbps	34 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	8	21	1 MB	27	1 MB	25.208396	0.4438	18 Mbps	21 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	9	21	1 MB	27	1 MB	25.210460	0.3131	26 Mbps	30 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	10	21	1 MB	27	1 MB	25.210575	0.4785	17 Mbps	20 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	11	21	1 MB	27	1 MB	25.210796	0.6265	13 Mbps	15 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	12	21	1 MB	27	1 MB	25.211001	0.5009	16 Mbps	18 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	13	21	1 MB	27	1 MB	25.211277	0.5903	14 Mbps	16 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	14	21	1 MB	27	1 MB	25.212038	0.6201	13 Mbps	15 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	15	21	1 MB	27	1 MB	25.212109	0.8112	13 Mbps	15 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	16	21	1 MB	27	1 MB	25.213430	0.5338	15 Mbps	17 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	48	1 MB	17	21	1 MB	27	1 MB	25.214573	0.6487	12 Mbps	14 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	50	1 MB	18	22	1 MB	28	1 MB	25.214847	0.4586	13 Mbps	15 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	50	1 MB	19	22	1 MB	28	1 MB	25.215094	0.6457	13 Mbps	15 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	30	1 MB	20	22	1 MB	28	1 MB	25.215309	0.3558	13 Mbps	15 Mbps	1
127.0.0.1	5005	127.0.0.1	5005	17	224 KB	21	9	224 KB	8	364 bytes	25.217007	0.4043	4480 kbps	7195 bits/s	1

Observations (From Wireshark)

Method	Run	Packets Sent	Retransmissions	Time Taken (sec) At Wireshark	Time Taken Output from Code
Single TCP	1	5152	0	10.412420-10.313809= 0.098611	0.10
Single TCP	2	5152	0	20.015232-19.915380= 0.099852	0.10
Single TCP	3	5152	0	19.153944- 19.0505125= 0.1034315	0.10
Multi TCP (4 conns)	1	495	0	7.466278-6.636331= 0.829947	0.829
Multi TCP (4 conns)	2	495	0	16.170756-15.387531= 0.783225	0.782
Multi TCP (4 conns)	3	495	0	25.863210-25.205345= 0.657865	0.656

*No. of packets = No. of packets sent from client to server.

Analysis -

1. Performance Observation

- For Single TCP Connection file transfers completion in approximately 0.1 seconds.
- The Multi TCP Connection took longer (0.65–0.83 seconds)

2. Reason for Observed behaviour

1. **Bandwidth competition:** Multiple TCP connections share the same network bandwidth, causing congestion and increased latency.
2. **Threading and synchronization overhead:** Managing several threads and sockets adds CPU and coordination delays on both client and server sides.
3. **Chunk reordering and merging delay:** The server must reorder and combine chunks from different connections, introducing extra processing time.
4. **Small file size:** For small files, the time spent setting up and closing multiple connections outweighs the benefits of parallel transfer.

Conclusion –

We get best performance for Single TCP Connection over Multiple TCP connections. As we can see for file with size small to medium single TCP stream provides lower overhead , best congestion control and transfer Fastly.

Multi TCP connections may be beneficial only for very large files or high-bandwidth networks where parallelism can outweigh setup overhead.

Please find attached code and pcap file -

<https://drive.google.com/drive/folders/1sTvcj6GpLwEGD7scxqco0EWz1r7bGkVs?usp=sharing>